All questions in the exam have one (1) and only one correct alternative.

Answering with more than one alternative for a question will result in zero (0) points for the question.

General instruction for understanding the code

Unless the question states differently, understand the code in this exam as:

1. Standard JavaScript code as of latest Ecmascript (ECMA262, ed. 15, 2024).
2. Each question is its own new empty tab in a web-browser (Chrome) and the code is in a `<script>`-tag in a html file loaded by the browser.

Questions

Section 1: JavaScript Basics

1. JavaScript is primarily considered a: `(1p)`

a) Compiled language
b) Interpreted language
c) Markup language

2. Which of the following are primitive data types in JavaScript? `(1p)`

a) `Boolean` and `Undefined`
b) `Array` and `Object`
c) `Function` and `Symbol`

3. The `this` keyword inside a regular function (note, not a class method) refers to: `(1p)`

a) The global object in non-strict mode
b) The function itself
c) Always the object that called the function

4. Which variable declaration is block-scoped in JavaScript? `(1p)`

a) `var`
b) `let`
c) `function`

5. What feature in JavaScript allows functions to have access to variables from an outer function scope even after the outer function has returned? `(1p)`

a) Promises
b) Closures/Lambdas
c) Hoisting

6. In a browser, the global object is represented by: `(1p)`

a) `document`
b) `globalThis`
c) `global`

7. JSON stands for: `(1p)`

a) JavaScript Object Notation
b) Java Source Object Notation
c) Java Standard Output Name

8. Which keyword is used to create a subclass in JavaScript classes? `(1p)`

a) `inherits`
b) `extends`
c) `implements`

9. JavaScript's execution model is: `(1p)`

a) Single-threaded and asynchronous
b) Multi-threaded and synchronous
c) Single-threaded and synchronous

10. In Javascript code running in Node.js, the global object is represented by: (1p)

a) `globalThis`
b) `window`
c) `document`

Section 2: Types and Type Conversion

11. Given a variable that has the value:

```
NaN
```

What is the resulting value when converted to a *`boolean`*? (2p)

1. `undefined`
2. `null`
3. `true`
4. `false`
5. `""`
6. `"NaN"`
7. `0`
8. `1`
9. `Infinity`
10. `-Infinity`
11. `NaN`

12. Given a variable that has the value:

```
""
```

What is the resulting value when converted to a *`number`*? (2p)

1. `undefined`
2. `null`
3. `true`
4. `false`
5. `""`
6. `"0"`
7. `0`
8. `1`
9. `Infinity`
10. `-Infinity`
11. `NaN`

13. Given a variable that has the value:

```
1
```

What is the resulting value when converted to a *boolean*? (2p)

1. undefined
2. null
3. true
4. false
5. ""
6. "1"
7. 0
8. 1
9. Infinity
10. -Infinity
11. NaN

14. Given a variable that has the value:

```
0
```

What is the resulting value when converted to a *boolean*? (2p)

1. undefined
2. null
3. true
4. false
5. ""
6. "0"
7. 0
8. 1
9. Infinity
10. -Infinity
11. NaN

Section 3: Functions

15. Given the following JavaScript code:

```
function sum(a, b) {
    return a + b
}
```

Which of the following will correctly call the function and return 5? (2p)

a) sum("2", "3")
b) sum(2, 3)

c) `sum(5)`
d) `sum([2, 3])`

16. Which of the following is an invalid way to declare a function in JavaScript? (2p)

a) `function myFunc() {}`
b) `let myFunc = function() {}`
c) `let myFunc = () => {}`
d) `function = myFunc() {}`

17. Given the following JavaScript function:

```
function greet(name = "Guest") {
    return `Hello, ${name}!`
}
```

What will the output be for `greet()`? (2p)

a) `Hello, undefined!`
b) `Hello, Guest!`
c) `Hello, !`
d) An error is thrown

18. Given the following JavaScript function:

```
function max(...numbers) {
    return Math.max(...numbers)
}
```

Which of the following will return the value `10`? (2p)

a) `max([2, 5, 10])`
b) `max(2, 5, 10, 11)`
c) `max({2, 5, 10})`
d) `max(...[2, 5, 10])`

19. Consider the following code:

```
let multiply = function(num) {
    return num * num
}

let foo = multiply
```

What will `foo(3)` return? (2p)

a) `6`

b) `27`
c) `3`
d) `9`

Section 4: Exceptions, throw, try/catch/finally

20. What will the following code output? (2p)

```
try {
    console.log("Start")
    throw new Error("Oops!")
    console.log("Middle")
} catch (e) {
    console.log("Caught")
} finally {
    console.log("End")
}
```

a) Start Middle Caught End
b) Start Caught End
c) Start End
d) Start Caught

21. What type of error will be thrown by the following code? (2p)

```
console.log(x)
```

Assuming ***x*** *has not been declared.*

a) ReferenceError
b) TypeError
c) SyntaxError
d) RangeError

Section 5: Strings

22. What will the following code return? (1p)

```
"Hello World".includes("World")
```

a) `false`
b) `"World"`
c) `true`
d) `undefined`

23. What is the result of the following code? (1p)

```
let text = " JavaScript "
console.log(text.trimStart())
```

a) " JavaScript"
b) " JavaScript "
c) "JavaScript"
d) "JavaScript "

24. What will the following code output? (1p)

```
"ABCDEFG".charAt(2)
```

a) C
b) D
c) E
d) F

25. What will be the output of the following code? (1p)

```
let numString = "42"
console.log(parseInt(numString) + 8)
```

a) "508"
b) 428
c) 50
d) NaN

Section 6: Loops

26. What will be the output of the following code? (2p)

```
let total = 0
for(let i = 1; i <= 5; i++) {
    if(i === 3) break
    total += i
}
console.log(total)
```

a) 6
b) 15
c) 9
d) 3

27. What will be the value of `result` after the following code executes? (2p)

```
let numbers = [1, 2, 3, 4, 5]
let result = ""
numbers.forEach(function(num) {
    if(num > 3){
        return
    }
    result += num
})
console.log(result)
```

a) ""
b) "45"
c) "123"
d) "12345"

28. What will the following code output? (2p)

```
outerLoop:
for(let i = 0; i < 3; i++){
    for(let j = 0; j < 3; j++){
        if(i === 1 && j === 1){
            break outerLoop
        }
        console.log(`${i},${j}`)
    }
}
```

a) 0,0 0,1
b) 0,0 0,1 0,2 1,0
c) 0,0 0,1 0,2 1,0 1,1
d) 0,0 0,1 0,2 1,0 2,0 2,1 2,2

29. What is the output of the following code? (2p)

```
let count = 0
while(count < 5) {
    count++
    if(count === 3) {
        continue
    }
    console.log(count)
}
```

a) 1 2 3 4 5
b) 1 2 3 4
c) 0 1 2 4
d) 1 2 4 5

30. What's the output of the following code? (2p)

```
let arr = [2, 4, 6, 8]
let sum = 0
arr.forEach(num => {
    if(num === 6) {
        return
    }
    sum += num
})
console.log(sum)
```

a) 10
b) 20
c) 12
d) 14

Section 7: Arrays and Their Functions

31. What will the following code output? (2p)

```
const nums = [1, 2, 3, 4]
console.log(nums[3])
```

a) 1
b) 2
c) 3
d) 4

32. What is the output of the following code? (2p)

```
const arr = [1, 2, 3]
arr.push(4)
console.log(arr)
```

a) [1, 2, 3, 4]
b) [4, 1, 2, 3]
c) [1, 2, 3]
d) [4]

33. What will the following code snippet output? (2p)

```
const data = [10, 20, 30, 40, 50]
console.log(data.slice(2, 4))
```

a) [10, 20, 30, 40]
b) [30, 40]
c) [30, 40, 50]
d) [20, 30]

34. What will be the output of the following code? (2p)

```
const items = [5, 10, 15, 20]
console.log(items.shift())
```

a) [5, 10, 15, 20]
b) 20
c) [10, 15, 20]
d) 5

Section 8: Objects

35. What is the output of the code below? (2p)

```
const user = {
    name: "Alice"
}
console.log(user["name"])
```

a) "name"
b) "Alice"
c) user
d) undefined

36. What is the output of the code below? (2p)

```
const pet = {
    species: "Cat"
}
console.log("age" in pet)
```

a) true
b) "Cat"
c) undefined
d) false

37. What is the output of the code below? (2p)

```
const book = {
    title: "1984",
    author: "George Orwell"
}
console.log(Object.keys(book)[0])
```

a) "author"
b) "1984"
c) "title"
d) "George Orwell"

38. What is the output of the code below? (2p)

```
const person = {
    name: "Bob",
    greet: function() {
        return "Hi, " + this.name
    }
}
const sayHi = person.greet
console.log(sayHi())
```

a) "Hi, "
b) "Hi, Bob"
c) "Hi, undefined"
d) "Hi, this.name"

39. What is the output of the code below? (2p)

```
const numbers = {
    a: 1,
    b: 2,
    sum() {
        return this.a + this.b
    }
}
console.log(numbers.sum())
```

a) 3
b) undefined
c) NaN
d) "1,2"

Section 9: Prototypes and Classes

40. What is the output of the code below? (2p)

```
function Vehicle(make) {
    this.make = make
}

Vehicle.prototype.start = function() {
    return `${this.make} is starting`
}

function Car(make) {
    Vehicle.call(this, make)
}

Car.prototype = Object.create(Vehicle.prototype)
Car.prototype.constructor = Car

Car.prototype.start = function() {
    return `${this.make} is driving`
}

let myCar = new Car("Honda")
console.log(myCar.start())
```

a) "Honda is driving"
b) "Honda is starting"
c) "Vehicle is starting"
d) "Car is driving"

41. What is the output of the code below? (2p)

```
class Rectangle {
    constructor(width, height) {
        this.width = width
        this.height = height
    }

    area() {
        return this.width * this.height
    }
}

let rect = new Rectangle(5, 4)
console.log(rect.area())
```

a) 9
b) 10
c) 20
d) Error

42. What is the output of the code below? (2p)

```
function Person(name) {
    this.name = name
}

Person.prototype.sayName = function() {
    return `My name is ${this.name}`
}

let person1 = new Person("Eve")
let person2 = Object.create(person1)

console.log(person2.sayName())
```

a) "My name is Eve"
b) "My name is "
c) "My name is undefined"
d) Error

Section 10: Iterators & Generators

43. What is the output of the code below? (2p)

```
let array = ['x', 'y', 'z']
let iterator = array.entries()

let output = iterator.next().value
console.log(output)
```

a) [1, 'y']
b) [0, 'x']
c) 'x'
d) [2, 'z']

44. What will be the output of the following code? (2p)

```
let letters = ['a', 'b', 'c']
let iterator = letters.keys()

let key = iterator.next().value
```

```
key = iterator.next().value
console.log(key)
```

a) 'a'
b) 'b'
c) 0
d) 1

45. What will be the output of the following code? (2p)

```
function* numberGen() {
    yield 1
    yield 2
    yield 3
}

let gen = numberGen()
let num = gen.next().value
num = gen.next().value
console.log(num)
```

a) undefined
b) 1
c) 2
d) 3

46. What will be the output of the following code? (2p)

```
function* idGenerator() {
    let id = 0
    while(true) {
        yield ++id
    }
}

let gen = idGenerator()
gen.next()
gen.next()
let id = gen.next().value
console.log(id)
```

a) 1
b) 2
c) 3
d) 4

Section 11: Modules

47. Consider the following files:

mathModule.js:

```
const pi = 3.14
export default pi
```

app.js:

```
import piValue from './mathModule.js'

console.log(piValue)
```

What will be the output when `app.js` is executed? `(2p)`

a) `3.14`
b) `pi`
c) `mathModule`
d) `undefined`

Section 12: Standard Library

48. What will be the output of the following code? `(1p)`

```
const arr = [1, 2, 3]
const values = arr.map(x => x * x)
console.log(values)
```

a) `[1, 2, 3]`
b) `[2, 4, 6]`
c) `[1, 4, 9]`
d) `undefined`

49. What will be the output of the following code? `(1p)`

```
const nums = [5, 10, 15]
const result = nums.reduce((sum, num) => sum + num, 0)
console.log(result)
```

a) `30`
b) `0`
c) `5`
d) `[5, 10, 15]`

50. What will be the output of the following code? (2p)

```
const arr = [2, 4, 6, 8]
const filtered = arr.filter(num => num > 5)
console.log(filtered)
```

a) [2, 4, 6, 8]
b) [6, 8]
c) [2, 4]
d) []

51. What will be the output of the following code? (2p)

```
const chars = ['x', 'y', 'z']
let str = ''
chars.forEach(char => str += char.toUpperCase())
console.log(str)
```

a) 'XYZ'
b) 'xyz'
c) ['X', 'Y', 'Z']
d) undefined